```
// Home.vue
export default {
  computed: {
    username() {
      // We will see what `params` is shortly
      return this.$route.params.username
    }
  },
  methods: {
    goBack() {
      window.history.length > 1 ? this.$router.go(-1) : this.$router.push('/')
    }
  }
}
```


Injecting the router

Keep in mind that using this.$router is the same as using a router. This is why we need it. Since we don't want to import the router in any single component that needs to manipulate routing, we use $router.

Observe that a <router-link> automatically gets the .router-link-active class when its target route is matched.

State Management

Official Flux-Like Implementation

Large applications often become more complicated as a result of many pieces of state spread through several components and their interactions. Vue provides vuex, an Elm-inspired state management library, to address this problem. It also interfaces with vue-devtools, allowing for zero-setup time travel debugging.

Simple State Management from Scratch

It is sometimes forgotten that the raw data object is the source of truth in Vue applications - a Vue instance merely proxies access to it. As a result, whenever you have a piece of state that needs to be shared by several instances, you will do it by identity.

When sourceOfTruth modifies, both vmA and vmB can immediately upgrade

```
var sourceOfTruth = {}

var vmA = new Vue({
  data: sourceOfTruth
})

var vmB = new Vue({
  data: sourceOfTruth
})
```


Simple State Management